



# Python 开发调试

打印状态



print()

# 打印状态



```
import random
```

```
def fun(s):
```

```
    n = random.randint(-s,s)
```

```
    print('DEBUG: n = %d' % n)
```

```
    return 100 / n
```

```
while True:
```

```
    fun(100)
```



# 优劣

- 干扰正常输出
- 代码需要清理
- 简单方便

断言



# assert

如果断言失败，assert语句会抛出AssertionError

# 断言



```
import random
```

```
def fun(s):
```

```
    n = random.randint(-s,s)
```

```
    print(' n = %d' % n)
```

```
    assert n != 0, 'n is zero!'
```

```
    return 100 / n
```

```
while True:
```

```
    fun(100)
```



# 优劣

- 异常仍然被触发
- 代码需要清理
- 异常是被自己知道的情况触发

日志



# logging



```
import random
import logging
logging.basicConfig(level=logging.INFO)
```

```
def fun(s):
    n = random.randint(-s,s)
    logging.info('INFO:n = %d' % n)
    logging.debug('DEBUG:n = %d' % n)
    return 100 / n
```

```
while True:
    fun(100)
```



```
import random
import logging
logging.basicConfig(level=logging.DEBUG)
```

```
def fun(s):
    n = random.randint(-s,s)
    logging.info('INFO:n = %d' % n)
    logging.debug('DEBUG:n = %d' % n)
    return 100 / n
```

```
while True:
    fun(100)
```



# 优劣

- 无需清理代码
- 分等级显示信息（在设置默认格式的时候设置的等级更高，那么低等级语句将不会输出）
- 信息可以指定模式（文件，console等等）

| Level    | Numeric value |
|----------|---------------|
| CRITICAL | 50            |
| ERROR    | 40            |
| WARNING  | 30            |
| INFO     | 20            |
| DEBUG    | 10            |
| NOTSET   | 0             |



```
import logging
import sys

def test_log_level():
    # set default logging configuration

    logger = logging.getLogger()    # initialize logging class
    logger.setLevel(logging.DEBUG)  # default log level

    format = logging.Formatter("%(asctime)s - %(message)s")    # output format
    sh = logging.StreamHandler(stream=sys.stdout)    # output to standard output
    sh.setFormatter(format)
    logger.addHandler(sh)

    # use logging to generate log ouput
    logger.info("this is info")
    logger.debug("this is debug")
    logger.warning("this is warning")
    logger.error("this is error")
    logger.critical("this is critical")

test_log_level()
```

工具



IDE



# IDLE

- 断点调试
- 右键设置

```
1 import random
2
3 def fun(s):
4     n = random.randint(-s,s)
5     return 100 / n
6
7 while True:
8     fun(100)
9
```

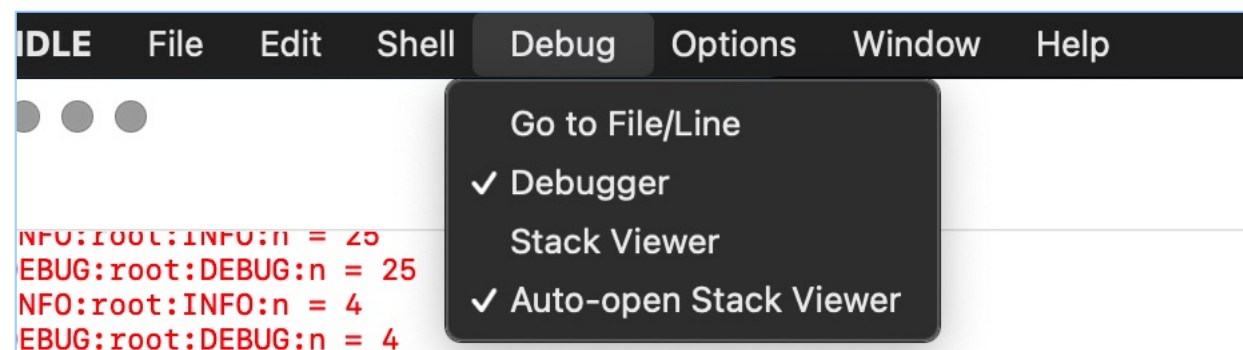
Cut  
Copy  
Paste  
Set Breakpoint  
Clear Breakpoint

```
1 import random
2
3 def fun(s):
4     n = random.randint(-s,s)
5     return 100 / n
6
7 while True:
8     fun(100)
9
```



# IDLE

- 开启debugger

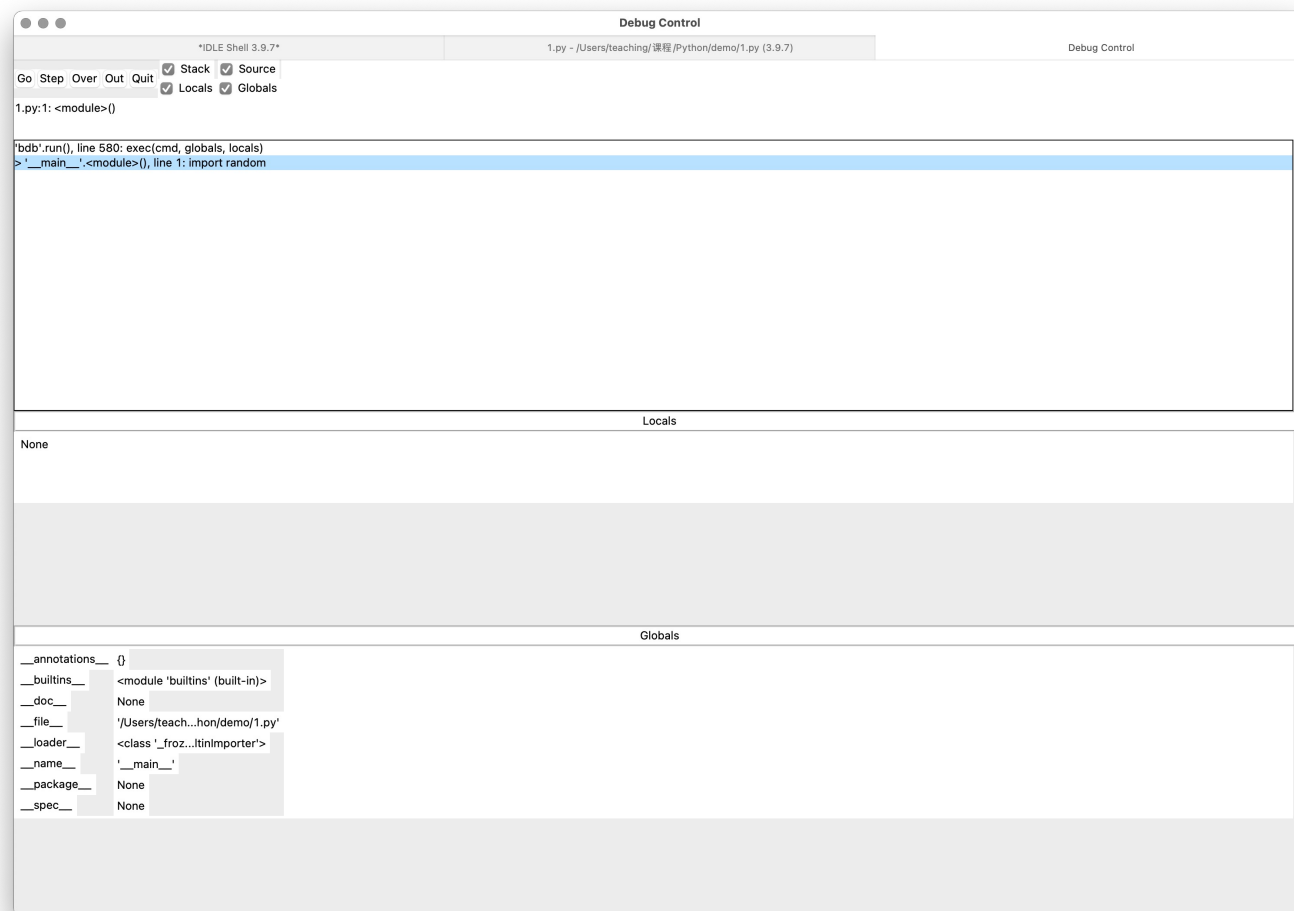


```
>>>  
[DEBUG ON]
```

# IDLE

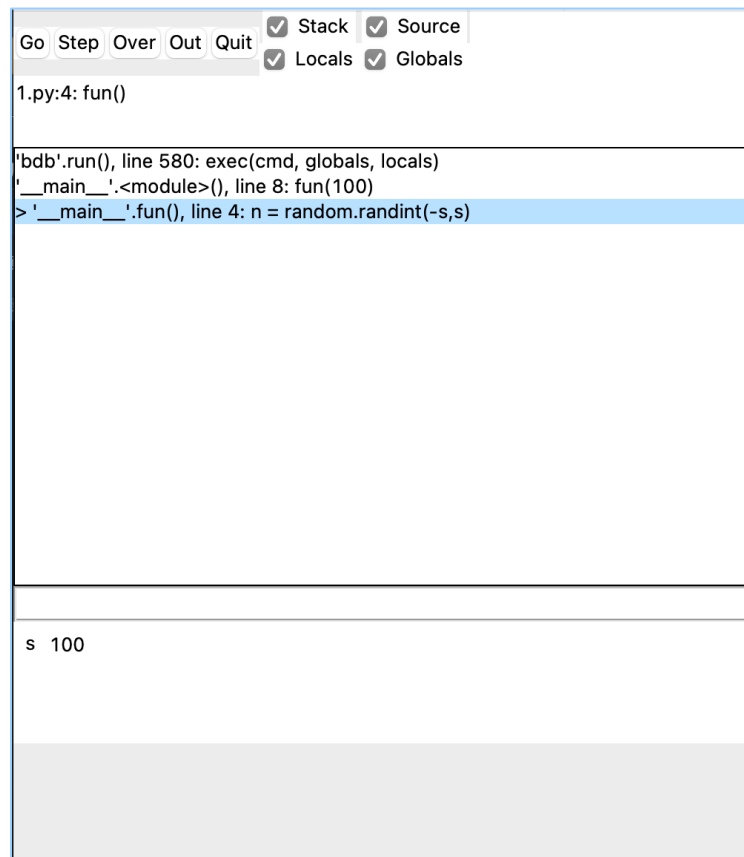


- 运行代码
- 当前行
- Local变量
- 全局变量



# GO

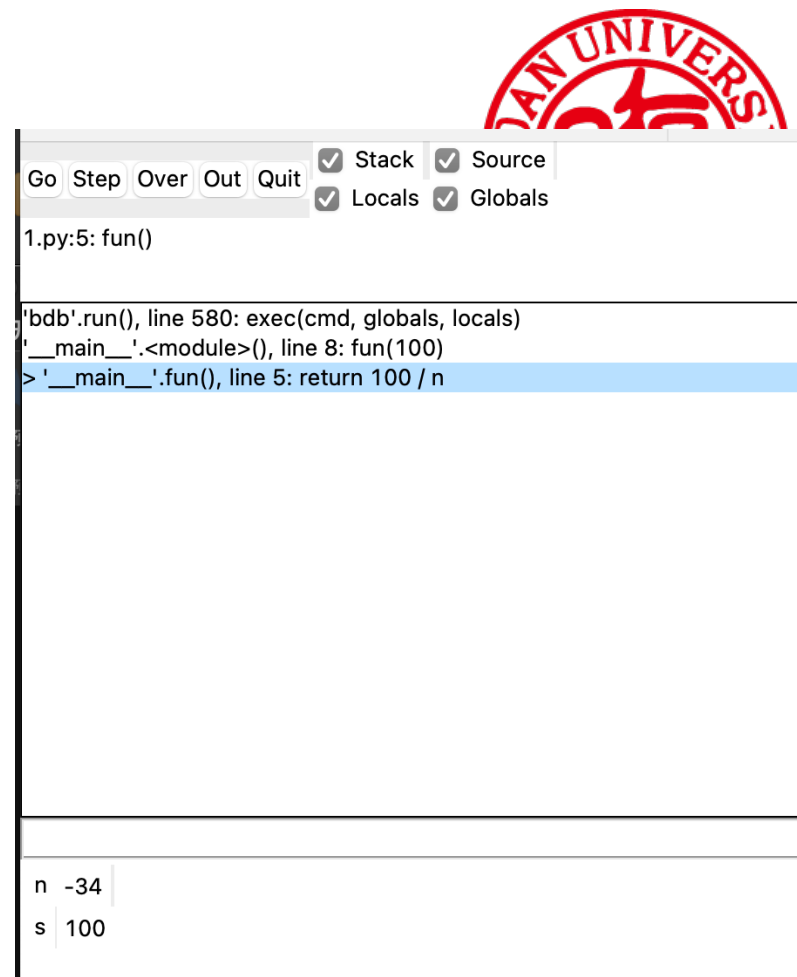
- 进行到断点位置
- 多次点击测试



```
Go Step Over Out Quit ☒ Stack ☒ Source
☒ Locals ☒ Globals
1.py:4: fun()

'bdb'.run(), line 580: exec(cmd, globals, locals)
'__main__'.<module>(), line 8: fun(100)
> '__main__.fun(), line 4: n = random.randint(-s,s)

s 100
```



```
Go Step Over Out Quit ☒ Stack ☒ Source
☒ Locals ☒ Globals
1.py:5: fun()

'bdb'.run(), line 580: exec(cmd, globals, locals)
'__main__'.<module>(), line 8: fun(100)
> '__main__.fun(), line 5: return 100 / n

n -34
s 100
```



## Step 模式

- Step into , 进入子函数运行
- Step over , 单步运行（会直接执行完该函数）在函数内遇到子函数时不会进入子函数内单步执行，而是将子函数整个执行完再停止，也就是把子函数整个作为一步
- Step out , 跳出子函数

# 中间变量的显示

```
1.py:5: fun()
```

```
'bdb'.run(), line 580: exec(cmd, globals, locals)
```

```
'__main__'.<module>(), line 8: fun(100)
```

```
> '__main__.fun(), line 5: return 100 / n
```

```
n 30
```

```
s 100
```



# PyCharm

Professional 2021.2

JET  
BRAINS



# Pycharm



```
demo - 1.py

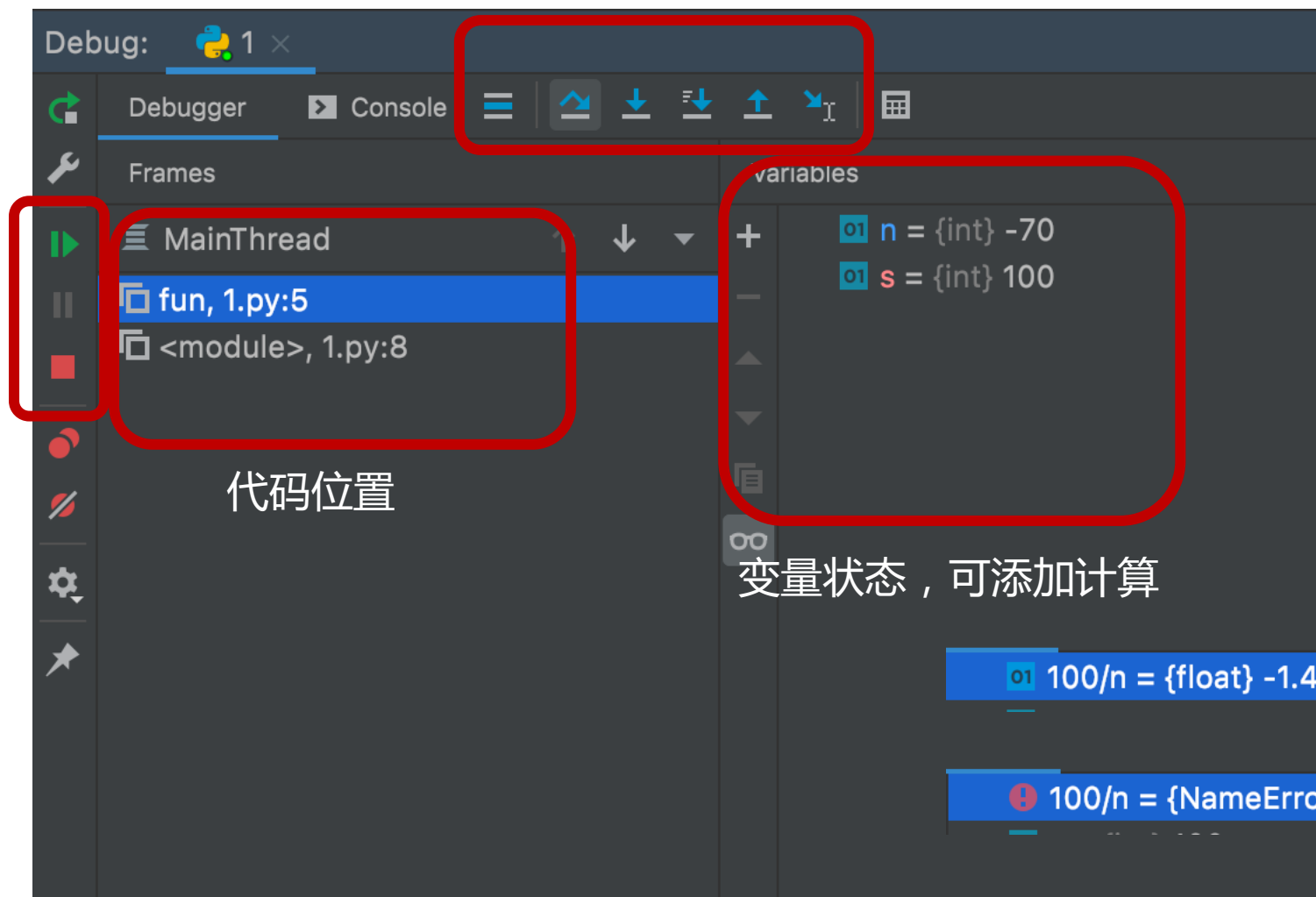
1  import random
2
3  def fun(s):
4      n = random.randint(-s,s)
5      return 100 / n
6
7  while True:
8      fun(100)
9
```



# 控制代码步骤

Step步进区域

断点执行





```
1  import random
2
3  def fun(s):    s: 100
4      n = random.randint(-s,s)    n: -70
5      return 100 / n
6
7  while True:
8      fun(100)
9
```



Debug: 1 x

Debugger Console

Variables Watches

```
> 1 __exception__ = {tuple: 3} (<class 'ZeroDivisionE
    01 n = {int} 0
    01 s = {int} 100
```

+ - ▲ ▼ [ ] ∞

! 100/n = {ZeroDivisionError}division by zero



谢谢